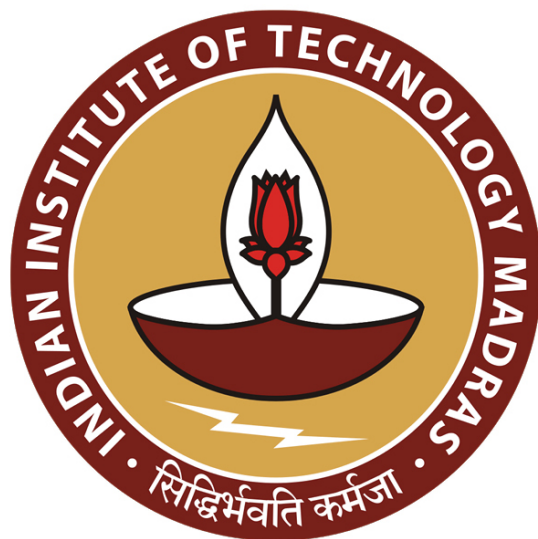
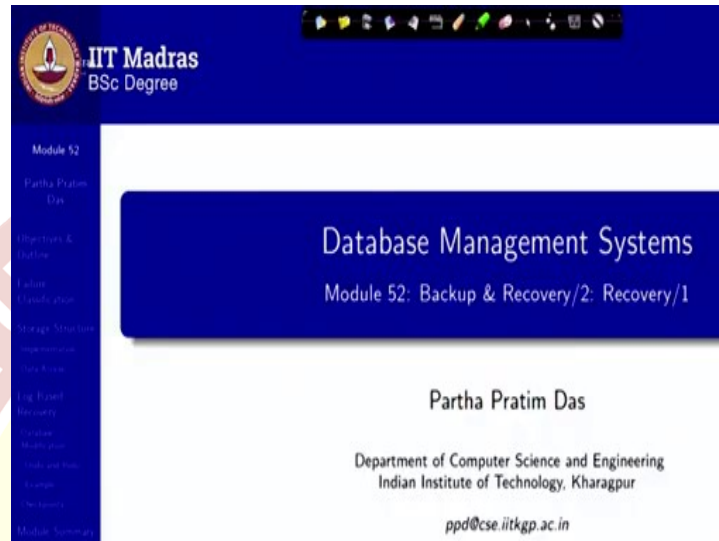




IIT Madras
ONLINE DEGREE

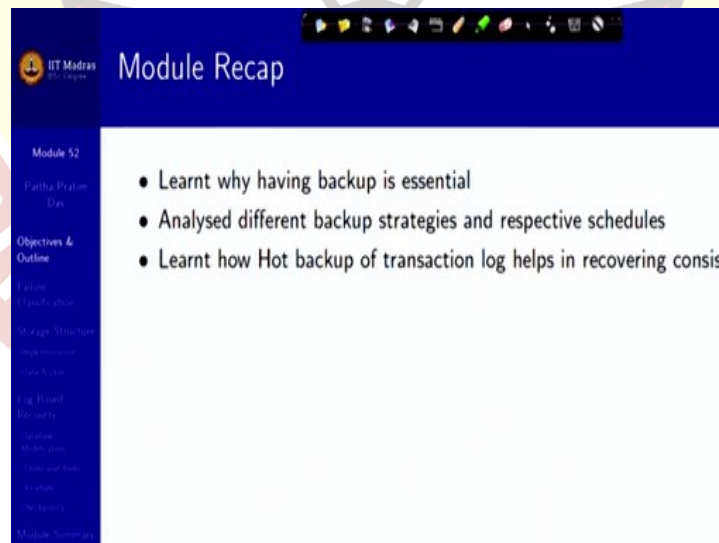
Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Backup & Recovery 2: Recovery 1

(Refer Slide Time: 0:27)



Welcome to module 52 of Database Management Systems course in the IIT Madras online BSc program.

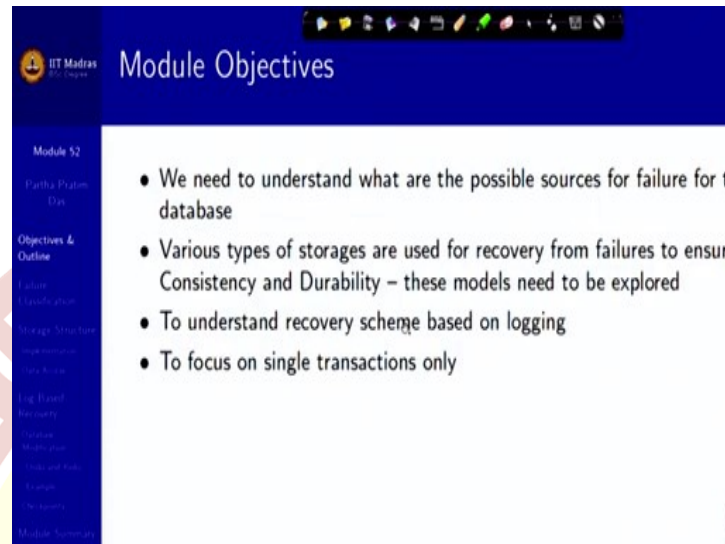
(Refer Slide Time: 0:33)



In the last module we have started talking about backup requirements, why backup is essential and analyzed different backup strategies and respective schedules. We notice that it is relatively easy to think about various cold backup strategies, but we also need hot backup

of transaction logs to help recovering from the failures at any point of time to keep the database consistent.

(Refer Slide Time: 1:05)



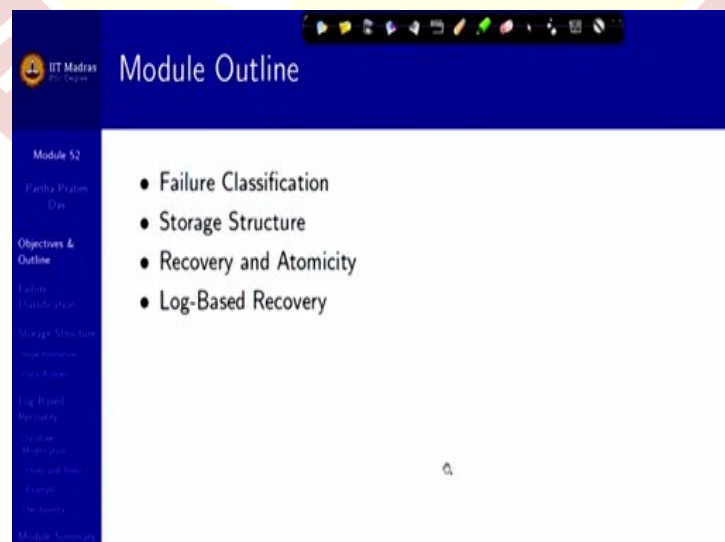
IIT Madras
P. Thirumalaiah
D. S. Srinivasan

Module Objectives

- We need to understand what are the possible sources for failure for transaction database
- Various types of storages are used for recovery from failures to ensure Consistency and Durability – these models need to be explored
- To understand recovery scheme based on logging
- To focus on single transactions only

So, we will start in this module discussing about the possible sources of failure for transactions in the database and the various strategies that can be used for recovery from these failures to keep the atomicity, consistency and durability in mind. And this will be primarily based on recovery schemes based on logging and please keep this in mind that this module will assume that at a time only a single transaction is working. In the next module we will take up the case of the similar situations when more one or, two or more transactions are in operation or the transactions are concurrent.

(Refer Slide Time: 1:52)



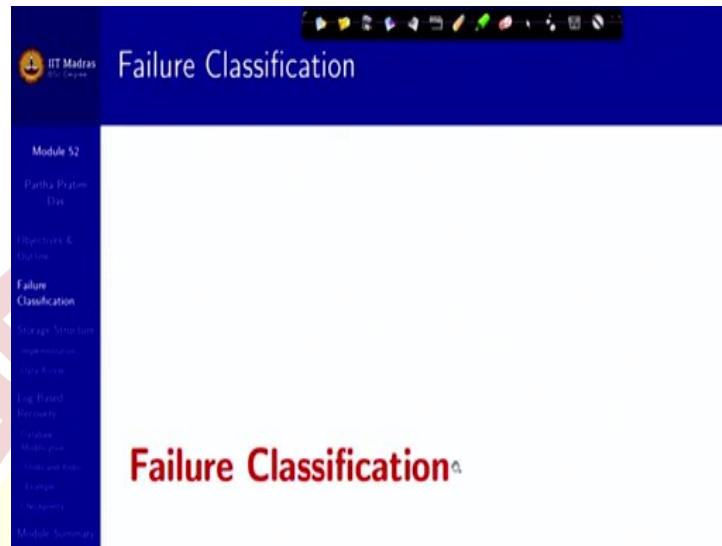
IIT Madras
P. Thirumalaiah
D. S. Srinivasan

Module Outline

- Failure Classification
- Storage Structure
- Recovery and Atomicity
- Log-Based Recovery

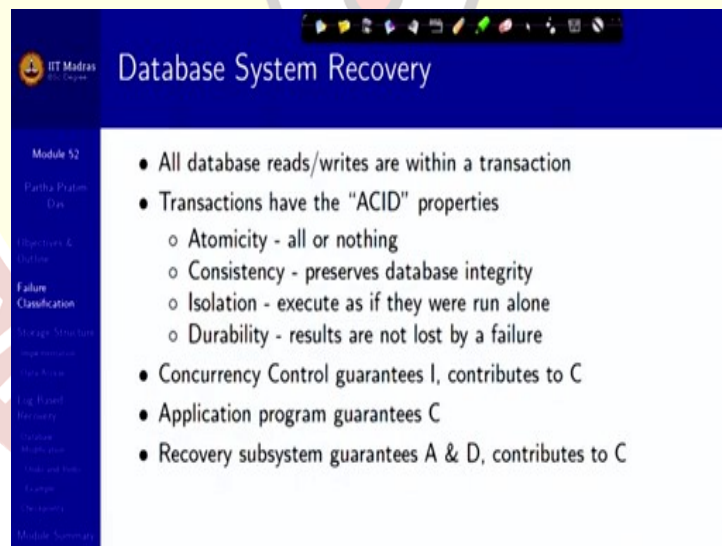
So, we will talk about failure classification, we will talk about certain types of storage also, recovery, atomicity and log-based recovery.

(Refer Slide Time: 2:02)



Failure classification the first thing.

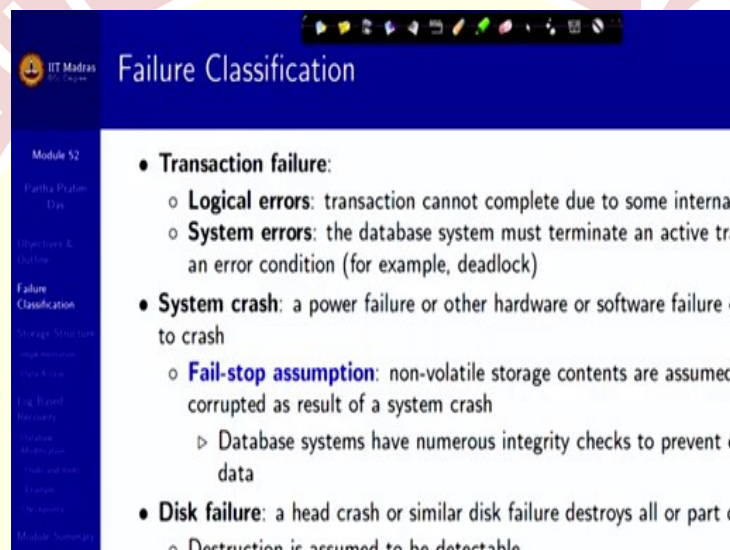
(Refer Slide Time: 2:07)



So, all databases do read/writes within a transaction we know and transactions have to follow ACID properties atomicity, all or nothing we have to guarantee, consistency which will preserve database integrity, isolation which ensures that transactions running concurrently must appear as if they are running alone, and durability which will result in a correct database in case of failure also, does not allow you to lose data.

So, concurrency control that we have done last week guarantees that we have isolation and thereby contributes towards consistency. Application programs also have to guarantee consistency if application program does some inconsistent logic like in terms of making a transfer of 50 rupees from account A to account B if it deducts 50 rupees from one account and credits 40 rupees to another account, database will become consistent and no common strategy can do anything, that is an application programming error, so it has to take care of that. And the recovery subsystems we are critical to guarantee the atomicity, durability, thereby contributing to the consistency that is the basic perspective that we are coming from.

(Refer Slide Time: 3:26)



So, failures can be of multiple types, transaction can fail because of logical errors the kind of errors I was talking of or system errors for example we have seen situations like the database system may terminate an active transaction because it has got into a deadlock, so these are the reasons the transaction can fail.

It could fail due to system crash you have a power failure, so that causes the system to crash and naturally fail stop assumption is good to make like non volatile storage contents are assumed to be not corrupted during the result of a system crash, it is considered that disks will not fail according to I mean due to the system crash, I mean there are lot of checks and balances for that.

But it can also fail for example a head crash or a similar disk failure will destroy all or part of the disk storage. This destruction is normally assumed to be detectable that it should be possible that this is going to happen or has happened and checksums will primarily detect this

kind of failure. So, these are the different types of transaction failure, system failure, disk failure, three main parts.

(Refer Slide Time: 4:46)

Recovery Algorithms

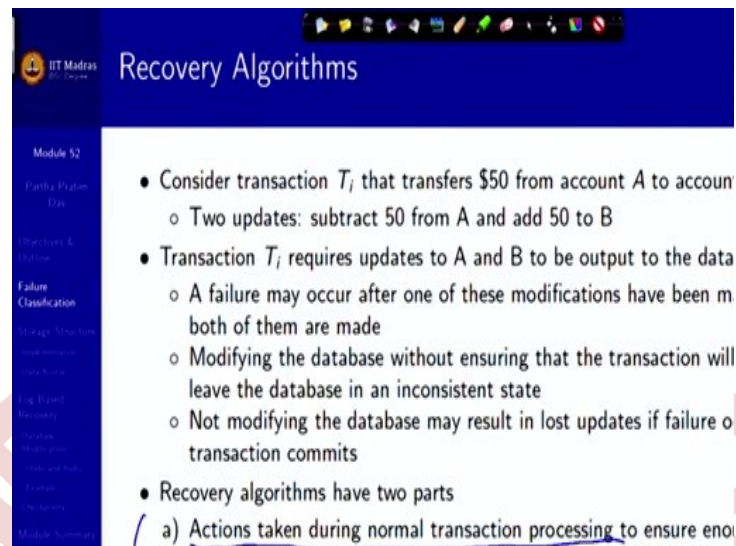
- Consider transaction T_i that transfers \$50 from account A to account B
 - Two updates: subtract 50 from A and add 50 to B
- Transaction T_i requires updates to A and B to be output to the data
 - A failure may occur after one of these modifications have been made, but before both of them are made
 - Modifying the database without ensuring that the transaction will leave the database in an inconsistent state
 - Not modifying the database may result in lost updates if failure occurs after transaction commits
- Recovery algorithms have two parts
 - a) Actions taken during normal transaction processing to ensure error recovery

Now, in case of a failure how do we recover. So, consider a transaction of data transfer. So, there are two updates, subtract 50 from A and add 50 to B. So, what are the things that can happen, that failure may occur after one of these modifications have been met, but before both have been met, if both are made then things are okay, but one has been done but the other has not been done.

Modifying the database ensuring that the transaction will commit leave the database in an inconsistent state in that case. Also not modifying the database may result in lost updates just after, if the failure occurs just after transaction commits, logically we are thinking the transaction has committed databases got updated, nice to think. But it is a finally a physical electronic system.

So, there is going to be several machine cycles between the transaction executing the commit and actually the data getting written in the persistent database. A failure can happen in between that. So, your view will be that transaction has committed but actually the data has not been modified.

(Refer Slide Time: 6:11)



Recovery Algorithms

- Consider transaction T_i that transfers \$50 from account A to account B
 - Two updates: subtract 50 from A and add 50 to B
- Transaction T_i requires updates to A and B to be output to the data
 - A failure may occur after one of these modifications have been made
 - Modifying the database without ensuring that the transaction will leave the database in an inconsistent state
 - Not modifying the database may result in lost updates if failure occurs before transaction commits
- Recovery algorithms have two parts
 - (a) Actions taken during normal transaction processing to ensure enough information to recover from failures

So, recovery are strategies for handling that and there are two parts to it, one part is you have to do certain things during the normal transaction processing so that you have enough information to recover from failures. See there is no magic, if you do not know what had gone in, what we are trying to do when you fail, then you will never be able to come back and correct that.

So, you have to take some actions to record enough information about what happened before the failure. And then the actions that are taken only if a failure occurs to recover the database based on the information that you have recorded during the normal transaction. And this will ensure the ACID properties of the transactions, particularly the atomicity, consistency and durability properties of the transactions. So, that is a, in principle that is a recovery strategy.

(Refer Slide Time: 7:11)



Now, before we go into I mean how we do that let us look at the storages we are dealing with. The first two types are known to you, volatile storage which evaporates if I have a system crash main memory, cache memory and so on. Nonvolatile storage which should survive the system crash, disk, tape, flash. Nonvolatile that is battery back, RAM or things like that. It may still fail losing the data but less frequently. So, it is that way more reliable I should say.

And then you have the concept of a stable storage, stable storage you can think of as a kind of a mythical concept of a storage where you say that it will survive all failures, no matter, I mean it is not a question of system crash, not a question of you know nonvolatile storage crash, it is not a question of disk head crash, nothing, it will survive everything. How do you have this kind of a mythical system?

You cannot have an ideally stable storage, but you can have an approximation to that by maintaining multiple copies on distinct nonvolatile media that is a simple thing, just have multiple copies and but there are issues with that. If you have multiple copies how do you update those copies, how do you make sure that one copy has may have gone bad, other copies are okay and so on but that is the basic concept of the stable storage.

(Refer Slide Time: 8:45)

Stable Storage Implementation

- Maintain multiple copies of each block on separate disks
 - copies can be at remote sites to protect against disasters such as fire or flooding
- Failure during data transfer can still result in inconsistent copies. Block transfer can:
 - Successful completion
 - Partial failure: destination block has incorrect information
 - Total failure: destination block was never updated
- Protecting storage media from failure during data transfer (one solution):
 - Execute output operation as follows (assuming two copies of each block):
 - ▷ Write the information onto the 1st physical block
 - ▷ When the 1st write is successful, write the same information onto the 2nd
 - ▷ The output is completed only after the second write successfully completes

Diagram: Secondary storage ↔ Local Recovery Manager ↔ Main memory (Fetch)

So, maintain multiple copies on separate disk. At times you have the copies at kept at multiple sites, different remote sites. For example, you may if you have three disk copies in the same building or in the same room then a fire happens or a flooding happens, earthquake happens and so usually depending on the criticality of the data you choose a remote site to keep this, obviously complications increase because the moment you have remote side you have much larger latency of data transfer between them and the question is how do you keep them updated.

Now, failure can still result in inconsistent copies. The block transfer that you have to do to multiple of them can have a successful completion, this is a happy thing which will happen most often. It can have a partial failure that is if you have been able to update but with the correct, incorrect information or it can have a total failure that you have not been able to update it at all. So, you will have to work with all of these practicalities in mind.

(Refer Slide Time: 10:01)

Stable Storage Implementation

- Maintain multiple copies of each block on separate disks
 - copies can be at remote sites to protect against disasters such as fire or flood
- Failure during data transfer can still result in inconsistent copies. Block transfer cases:
 - Successful completion
 - Partial failure: destination block has incorrect information
 - Total failure: destination block was never updated
- Protecting storage media from failure during data transfer (one solution):
 - Execute output operation as follows (assuming two copies of each block):
 - ▷ Write the information onto the 1st physical block
 - ▷ When the 1st write is successful, write the same information onto the 2nd
 - ▷ The output is completed only after the second write successfully completes

Diagram: Secondary storage, Local Recovery Manager, Main memory, Fetch.

So, typical output operation assuming that lower below I have tried to just give you a schematic diagram so you have the main memory in which a local recovery manager is working which use the database buffer manager we talked about earlier and it has the database buffers which are volatile.

And then you have a secondary storage which is a stable database let us say, so does not fail, takes care of through multiple copies. So, when you are assuming that you are say writing two copies of each block there could be more I mean more copies you keep naturally your resilience against failure increases, naturally the complexity of managing that becomes higher.

(Refer Slide Time: 10:54)

Stable Storage Implementation

- Maintain multiple copies of each block on separate disks
 - copies can be at remote sites to protect against disasters such as fire or flood
- Failure during data transfer can still result in inconsistent copies. Block transfer cases:
 - Successful completion
 - Partial failure: destination block has incorrect information
 - Total failure: destination block was never updated
- Protecting storage media from failure during data transfer (one solution):
 - Execute output operation as follows (assuming two copies of each block):
 - ▷ Write the information onto the 1st physical block
 - ▷ When the 1st write is successful, write the same information onto the 2nd
 - ▷ The output is completed only after the second write successfully completes

Diagram: Secondary storage, Local Recovery Manager, Main memory, Fetch.

But if you assume two copies then you write the information on to the first physical block. If the first write is successful then write the same information in the second physical block, you write one then write the next. And the output is considered complete only after the second write successfully completes. This is what is a basic strategy, I mean very native naive strategy I should say but that is what works best in these situations.

(Refer Slide Time: 11:28)

IIT Madras
Stable Storage Implementation (2)

Module 52
Parallel Systems
Disks

Protecting storage media from failure during data transfer (cont.):

- Copies of a block may differ due to failure during output operation
- To recover from failure:
 - First find inconsistent blocks:
 - ▷ Expensive solution : Compare the two copies of every disk block
 - ▷ Better solution:
 - Record in-progress disk writes on non-volatile storage (Not a special area of disk)
 - Use this information during recovery to find blocks that are inconsistent, and only compare copies of these
 - Used in hardware RAID systems

So, what can happen if you do that the copies of a block may differ due to a failure in the output operation. So, how do you recover from that? You can, you have to first find the inconsistent blocks. So, you can compare two copies of every disc block, then you will know what all are inconsistent that is excessively expensive, so you will not do that.

(Refer Slide Time: 11:59)

IIT Madras
Stable Storage Implementation (2)

Module 52
Parallel Systems
Disks

Protecting storage media from failure during data transfer (cont.):

- Copies of a block may differ due to failure during output operation
- To recover from failure:
 - First find inconsistent blocks:
 - ▷ Expensive solution : Compare the two copies of every disk block
 - ▷ Better solution:
 - Record in-progress disk writes on non-volatile storage (Not a special area of disk)
 - Use this information during recovery to find blocks that are inconsistent, and only compare copies of these
 - Used in hardware RAID systems

So, a better solution is record in progress disk writes on nonvolatile storage. As the disk writes are happening in a nonvolatile storage like a RAM or special area of the disk, you keep a record of how the disk writes are going on. Use this information when during recovery to find blocks that may be inconsistent.

Because there you have recorded what, if, when you have written if they are identical they were not. So, when you have to recover you just pick up those where you recorded that they are not identical. And this is a strategy which is used in the RAID systems, you will, we will discuss more about that, we have almost a full module discussing this later on.

(Refer Slide Time: 13:01)

IIT Madras

Stable Storage Implementation (2)

Module 52

Protecting storage media from failure during data transfer (cont.):

- Copies of a block may differ due to failure during output operation
- To recover from failure:
 - First find inconsistent blocks:
 - ▷ *Expensive solution* : Compare the two copies of every disk block
 - ▷ *Better solution*:
 - Record in-progress disk writes on non-volatile storage (Not a special area of disk)
 - Use this information during recovery to find blocks that may be inconsistent, and only compare copies of these
 - Used in hardware RAID systems

Now, finding that they are not consistent if either copy is inconsistent and is found to have an error how do you know an error, it has a bad checksum, we had talked about this that the hash value is kept as a checksum. So, if a either copy of the inconsistent block have an error then you overwrite, then you know that this is the erroneous blocks overwrite with the other. If both are correct but different then overwrite the second block by the first block as simple. So, this is the basic strategy that you can take with the stable storage being available to you.

(Refer Slide Time: 13:51)

Data Access

- **Physical Blocks** are those blocks residing on the disk
- **System Buffer Blocks** are the blocks residing temporarily in main memory
- Block movements between disk and main memory are initiated through two operations:
 - **input(B)** transfers the physical block B to main memory
 - **output(B)** transfers the buffer block B to the disk, and replaces the physical block there
- We assume, for simplicity, that each data item fits in, and is stored in, one block

Now, to make this more manageable and also to improve the throughput you do some more tricks. What you do is you have physical blocks which we talked about, you have system buffer blocks that are residing temporarily in the memory. So, physical blocks are being brought to the main memory buffer blocks and brought back so input process, output process.

(Refer Slide Time: 14:35)

Data Access (2)

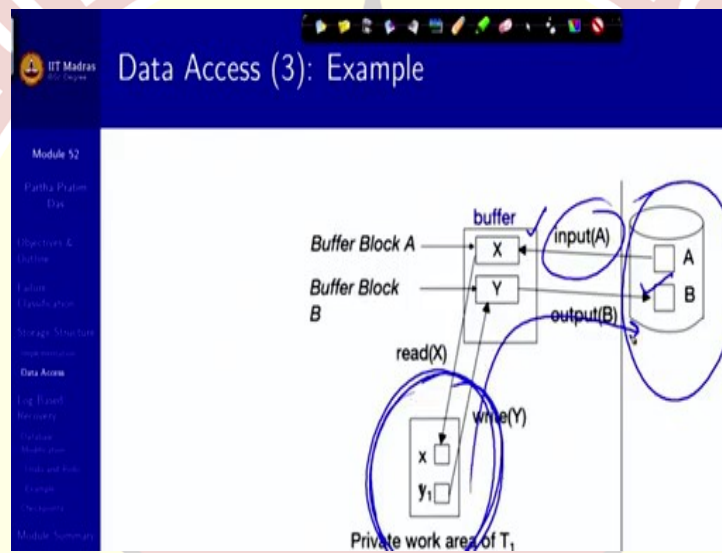
- Each transaction T_i has its private work-area in which local copies of accessed and updated by it are kept
 - T_i 's local copy of a data item X is denoted by x_i
 - B_X denotes block containing X
- Transferring data items between system buffer blocks and its private work-area
 - **read(X)** assigns the value of data item X to the local variable x_i
 - **write(X)** assigns the value of local variable x_i to data item X in the system buffer block
- Transactions
 - Must perform **read(X)** before accessing X for the first time (subsequent accesses can be from local copy)
 - The **write(X)** can be executed at any time before the transaction terminates

Now, when the transaction has to use it does not actually directly use the system buffer blocks rather the transaction will have its own local buffer that is different from the system buffer. So, what the transaction do is if they have, if the transaction T_i has to work on a data item x , it copies it from the system buffer to its local buffer, let us call it x_i . So, it has to if is

doing multiple reads then for the first time it has to read from the system buffer after that it can read from its own buffer, so that is a read X operation.

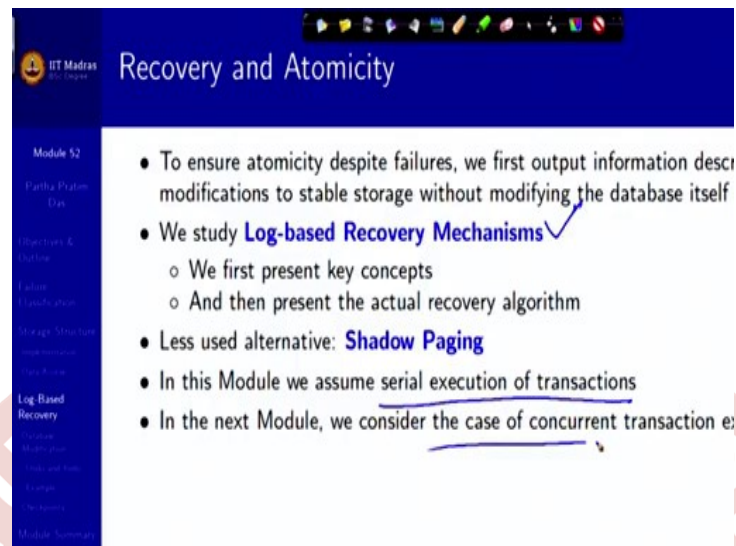
And for write X, you will write back this local value back onto the buffer block, so this is the, this is, so the transaction will keep on interacting with its own buffer only. And in case of read it will go to the system buffer, bring it to its own buffer. In case of write, it will write its value from the local buffer to the system buffer. And then you will have a output Bx, output Bx means outputting the system buffer where X was, the value of X was updated through write X back to the physical block.

(Refer Slide Time: 15:49)



So, schematically if we see it is very easy to understand, this is your physical blocks, input brings it to a buffer, output writes it from the buffer to the disk and this is a private work area, private buffer for the transaction and you make a copy, keep on using it here, compute new values and then you write back through the system buffer to the disk eventually, so this is the data access protocol that was typically used.

(Refer Slide Time: 16:26)

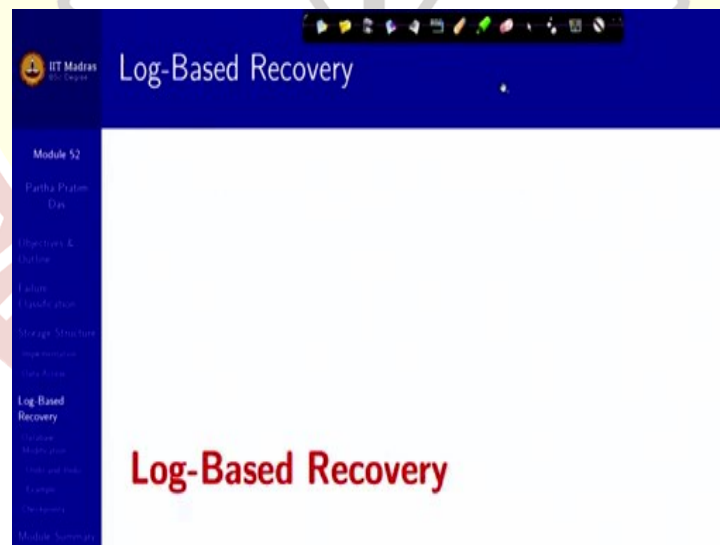


Recovery and Atomicity

- To ensure atomicity despite failures, we first output information describing modifications to stable storage without modifying the database itself
- We study **Log-based Recovery Mechanisms**
 - We first present key concepts
 - And then present the actual recovery algorithm
- Less used alternative: **Shadow Paging**
- In this Module we assume serial execution of transactions
- In the next Module, we consider the case of concurrent transactions

So, with this we will try to make sure that we can have a pretty robust recovery mechanism. So, for that what is used called a log-based recovery. As I was talking of in the last module also, a log-based recovery is the basic concept is that anytime you do an update you basically write down that update in a transaction log. Here we will show this mechanism for serial transactions, in the next module we will come with the concurrent transactions.

(Refer Slide Time: 17:04)



Log-Based Recovery

(Refer Slide Time: 18:26)

Log-Based Recovery

- A **log** is kept on stable storage
 - The log is a sequence of **log records**, which maintains information activities on the database
- When transaction T_i starts, it registers itself by writing a record $\langle T_i \rangle$
- Before T_i executes **write(X)**, a log record $\langle T_i, X, V_1, V_2 \rangle$ is written where V_1 is the value of X before the write (**old value**), and V_2 is the value to be written (**new value**)
- When T_i finishes its last statement, the log record $\langle T_i \text{ commit} \rangle$ is written

Diagram: Secondary storage (Stable log) and Main memory (Local Recovery Manager, Log buffers)

But in the transaction log you are keeping a record of what it has changed from, it has changed to V2, but you are keeping a record of the fact that it is changing from V1, which means that if you have to undo somehow roll back the transaction, roll back this write X then you will be able to know that the previous value of X was V1.

(Refer Slide Time: 18:53)

Log-Based Recovery

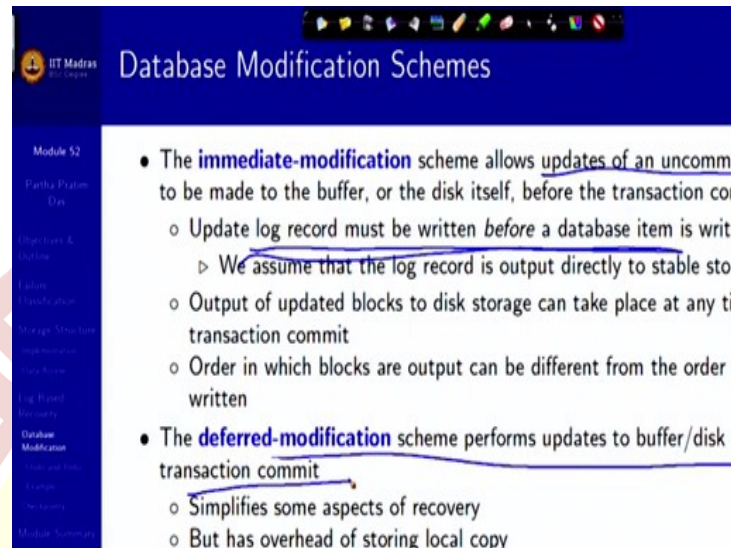
- A **log** is kept on stable storage
 - The log is a sequence of **log records**, which maintains information activities on the database
- When transaction T_i starts, it registers itself by writing a record $\langle T_i \rangle$
- Before T_i executes **write(X)**, a log record $\langle T_i, X, V_1, V_2 \rangle$ is written where V_1 is the value of X before the write (**old value**), and V_2 is the value to be written (**new value**)
- When T_i finishes its last statement, the log record $\langle T_i \text{ commit} \rangle$ is written

Diagram: Secondary storage (Stable log) and Main memory (Local Recovery Manager, Log buffers)

When the transaction finishes the last statement you write transaction commit, T_i commit this is the basic structure again. Here these things are the same in the memory, this also is the same but you have along with your database buffers, now you have the log buffers that you write to. And in addition to your stable database now you have in the secondary a stable log.

So, you are keeping a general entry for everything that you are doing an audit trail for everything that you are doing.

(Refer Slide Time: 19:24)



Database Modification Schemes

- The **immediate-modification** scheme allows updates of an uncommitted transaction to be made to the buffer, or the disk itself, before the transaction commit.
 - Update log record must be written *before* a database item is written to disk.
 - ▷ We assume that the log record is output directly to stable storage.
 - Output of updated blocks to disk storage can take place at any time before or after the transaction commit.
 - Order in which blocks are output can be different from the order in which they were written.
- The **deferred-modification** scheme performs updates to buffer/disk only at the time of transaction commit.
 - Simplifies some aspects of recovery.
 - But has overhead of storing local copy.

Now, the question naturally is that with this how do you make the actual database modifications? There are two schemes, one is immediate modification which allows updates of an uncommitted transaction to be made to the buffer or to the disk itself before the transaction commits, when the value is available it could.

So, as soon as the log record has been created, we assumed that the log record is assumed directly output to the stable storage and the update of the blocks of the disk storage or tape can any time before the transaction commits or even after the transaction commits. Deferred modification performs updates only at the time of transaction commit. So, this simplifies some aspects of the recovery but has a overhead of storing a large number of local copy because you are not updating anything during the transaction. So, for a big transaction this has a overhead of local copies.

(Refer Slide Time: 20:39)

Database Modification Schemes

- The **immediate-modification** scheme allows updates of an uncommitted transaction to be made to the buffer, or the disk itself, before the transaction commit
 - Update log record must be written *before* a database item is written to disk
 - ▷ We assume that the log record is output directly to stable storage
 - Output of updated blocks to disk storage can take place at any time after the transaction commit
 - Order in which blocks are output can be different from the order in which they were written
- The **deferred-modification** scheme performs updates to buffer/disk only after the transaction commit
 - Simplifies some aspects of recovery
 - But has overhead of storing local copy

So, we will in this course talk only about the first case that is the immediate modification scheme.

(Refer Slide Time: 20:46)

Transaction Commit

- A transaction is said to have committed when its commit log record is output to stable storage
 - All previous log records of the transaction must have been output already
- Writes performed by a transaction may still be in the buffer when the transaction commits, and may be output later

So, let us, so transaction commit you understand, a transaction is set to have committed now we are getting little bit fine grained. When is a transaction committed? When its commit log record is output to the stable storage. Because its values that have been generated, that have been written may or may not have been updated in the final buffers and disk.

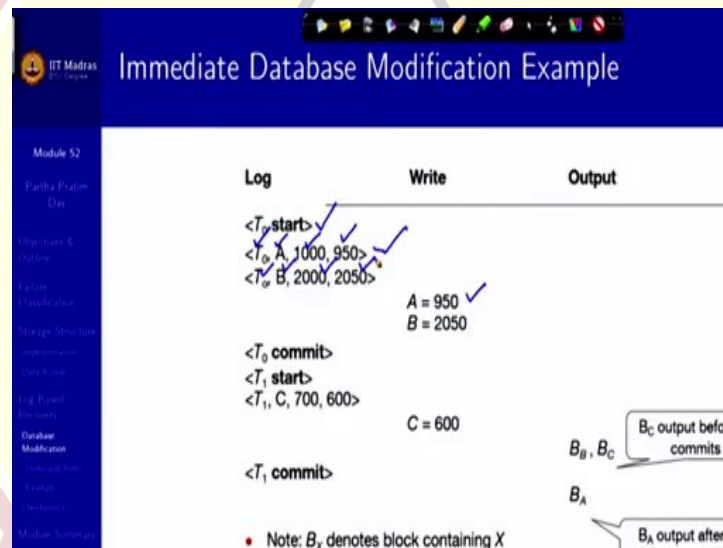
But its commit log record to the output storage, stable storage must be complete. So, that there is a record of the fact that the transaction has committed. And that also needs that all previous log records of the transaction must have been output already. You are doing

different kinds of buffering in between, so the actual time when these writes in the final stable database happens of the values and so on are not in your control.

But what is in your control is the sequence of actions that you are taking in the transaction, making sure that they are recorded in that order in the transaction block. And only after all those recording when your commit log record is put output to the stable storage you actually say that that is the point the transaction has committed.

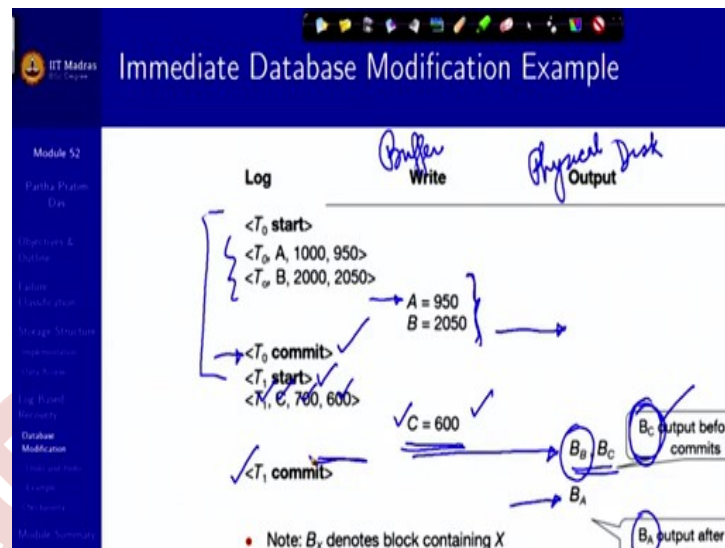
Whereas the writes performed by this transaction may still be in the buffer, may be output later on. So, keeping this in condition you will have to think about the logs, keeping this in mind you will have to think about what could be the situation when a failure happens and accordingly what you will need to do for the recovery.

(Refer Slide Time: 22:57)



So, let us look at an example for this basic log process. So, a transaction has started T₀, so it says start then it is writing 950 to A. So, and before writing that it is writing the transaction log assuming that A was 1000, it has written a log record saying that T₀ has is writing A changing its value from 1000 to 950. T₀ is writing B changing its value from 2000 to 2050. Together what does it mean? 50 rupees have been transferred from A to B.

(Refer Slide Time: 23:50)



After these logs have been written, the actual writes are done. So, what will this write do? This write will try to take the value from the transactions buffer to the system buffer, subsequently to the disk. And then the transaction commits by writing this log record. The output so this is, this write is taking it to the buffer and this is your output is basically taking it to the physical disk.

So, you can see that this still is in the buffer but you have committed, you have taken that transaction has taken place. Because you have in the log enough information to reconstruct the scenario at this point. Then another transaction T1 starts then T1 changes C from 700 to 600, T1 writes C 600 into the buffer.

And at this point, at this point BB and BC, what is BB? BB is a block, system block containing the value of the variable B. BC is a system block containing the value of the variable C output to the disk. So, what do you see? In case of C the BC has been written to the disk even before T1, has committed has been written.

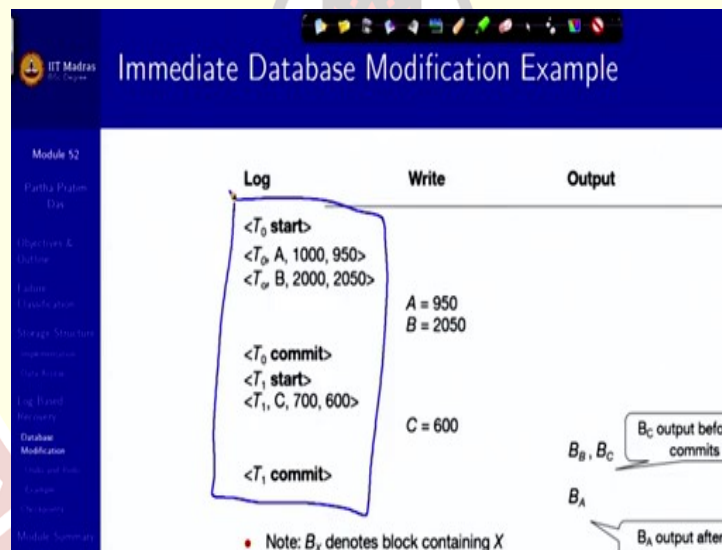
And BB is being written after T0 has committed, so for some it could be before the commit, for some it could be after the commit. Then T1 commits that it writes that, it has the log record T1 commit and then you have BA being output so BA is also output after T0 is commit. So, you can see that this dissociates, this ability to write down what is going on in terms of the transaction on the transaction log helps you to dissociate the process of, otherwise what you would have required is after you have done this you would have required to wait till this output to the disk happens and output to the disk you know is more expensive.

And one block may have multiple transactions, variables and it is up to the database to decide as to what is the appropriate time to write a buffer block back into the physical block. And if you have to wait for that in your transaction then your commit will get delayed your whole throughput will go down.

Similarly, if you do not allow something that you have already generated for which you have logically done a write, if you do not allow that to be written to the disk write in here then you cannot take advantage of the fact that possibly BC is being written at this point of time because it already has several changes of several variables due to other transactions.

So, if BC, if you do not allow BC to be written before T1 commits you will delay the output process. So, the first one delays the transaction commit process, the second one delays the output process.

(Refer Slide Time: 27:53)



But using the entire of this log mechanism you have got the freedom because it does not matter in which order the write you exactly know what logically has been done. So, that is the basic beauty of the mechanism.

(Refer Slide Time: 28:10)

Undo and Redo Operations

- **Undo** of a log record $\langle T_i, X, V_1, V_2 \rangle$ writes the **old** value V_1 to X
- **Redo** of a log record $\langle T_i, X, V_1, V_2 \rangle$ writes the **new** value V_2 to X
- **Undo and Redo of Transactions**
 - **undo(T_i)** restores the value of all data items updated by T_i to the value going backwards from the last log record for T_i
 - ▷ Each time a data item X is restored to its old value V a special log record (called **redo-only**) $\langle T_i, X, V \rangle$ is written out
 - ▷ When undo of a transaction is complete, a log record $\langle T_i, \text{aborted} \rangle$ is written out (to indicate that the undo was completed)
 - **redo(T_i)** sets the value of all data items updated by T_i to the new value going forward from the first log record for T_i

◦ No logging is done in this case

So, there are two other operations that we can do now. So, this is in a normal operation what will happen. So, one is undo log of a record, undo log will do the reverse, it will write the old value back to X . Redo log will write a new V_2 to X . So, what are the redo undo transaction? An undo transaction restores the value of all data items updated to T_i from their old value going backwards, going backwards from the point it is from the last log record.

Because log records have been done forward, just think about it very, very, I mean undo redo is something we keep on doing on our regular basis suppose you are editing a file in Word. You are typing, typing, typing and you have made mistakes, what you do, control Z. What is control Z is undo, that changes the last, that undoes the last changes done.

You see that the previous one also wrong, you can do another undo that does the previous one. So, you have as if going back, so what does, what do? It keeps a log of what all changes you are writing. And every time you do control Z undo, it just goes back and applies that gets you back to that previous value, you have deleted a word, it gets back the old word, that you have deleted and written a new one, so that is getting back the old value.

And after having done control Z for 5 steps, you say okay the fifth step you did not want to undo, mistakenly you have done undo, so what do you do? You do control Y. So, this is, control Y is redo, so it reapplies what you have already undone. So, this is exactly the same thing obviously when you do redo you do not do a logging because its already there in your previous log.

(Refer Slide Time: 30:25)

Undo and Redo Operations (2)

- The **undo** and **redo** operations are used in several different circumstances
 - The **undo** is used for transaction rollback during normal operation
 - ▷ in case a transaction cannot complete its execution due to sor
 - The **undo** and **redo** operations are used during recovery from failure
- We need to deal with the case where during recovery from failure and prior to the system having fully recovered

So, it can be used in two different circumstances. When one is undo is used for transaction roll back during normal operation, you are doing a normal operation and you have a roll back. So, you will always need to do undo, you will not need redo. But when you need to do recovery from failure you will need to do undo as well as redo, because certain things which have got lost, which were done but have got lost have to be redone so you need to do redo to be fully recovered.

(Refer Slide Time: 31:08)

Undo and Redo on Normal Transaction Rollback

- Let T_i be the transaction to be rolled back
- Scan log backwards from the end, and for each log record of T_i of the form $\langle T_i, X_j, V_1, V_2 \rangle$
 - Perform the undo by writing V_1 to X_j
 - Write a log record $\langle T_i, X_j, V_1 \rangle$ ✓
 - ▷ such log records are called **Compensation Log Records**
- Once the record $\langle T_i \text{ start} \rangle$ is found stop the scan and write the log record $\langle \text{abort} \rangle$

So, in a normal transaction, transaction to be rolled back, scan lock backward from the end, perform undo by writing V_1 to X write the log record, this is called the compensation log record, that you have undone. So, you will also have to create that requirement. And how

long would you go? You will go up to the beginning that is the reason you wrote where T_i started.

So, when you reach this going backwards when you reach T_i start, you know everything has been undone. So, in the log now you write, so you are writing compensation logs, now in the log you write T_i abort, because it has been aborted. So, this is the basic.

(Refer Slide Time: 32:00)

Module 52

Undo and Redo on Recovering from Failure

- When recovering after failure:
 - Transaction T_i needs to be undone if the log
 - ▷ contains the record $\langle T_i \text{ start} \rangle$,
 - ▷ but does not contain either the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$
 - Transaction T_i needs to be redone if the log
 - ▷ contains the records $\langle T_i \text{ start} \rangle$
 - ▷ and contains the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$
 - It may seem strange to redo transaction T_i if the record $\langle T_i \text{ abort} \rangle$ is in the log
 - ▷ To see why this works, note that if $\langle T_i \text{ abort} \rangle$ is in the log, the undo operation will write redo-only records. Thus, the redo operation will redo T_i 's modifications in this case. This slight redundancy is necessary to ensure recovery.

Now, when you are doing it during failure the transaction T_i needs to be undone if the log contains the record T_i start, but does not contain the T_i commit or T_i abort. Then you have to do the undo because it is partially done. But if we have to do, you have to redo this if it contains T_i start as well as T_i commit or T_i abort.

So, first case is it has not completed the whole thing. So, you know that changes have not fully happened, so you just do undo. But if you have start as well as end which is start as well as commit or abort whichever is the case then we will have to redo that whole thing to get the effect back and this is a process which is called repeating the history.

(Refer Slide Time: 32:57)

Immediate Modification Recovery Example

Below we show the log as it appears at three instances of time.

(a) <pre> <T₀ start> <T₀, A, 1000, 950> <T₀, B, 2000, 2050> </pre>	(b) <pre> <T₀ start> <T₀, A, 1000, 950> <T₀, B, 2000, 2050> <T₀ commit> <T₁ start> <T₁, C, 700, 600> </pre>	(c) <pre> <T₀ start> <T₀, A, 1000, 950> <T₀, B, 2000, 2050> <T₀ commit> <T₁ start> <T₁, C, 700, 600> <T₁ commit> </pre>
--	--	---

Recovery actions in each case above are:

(a) undo (T₀): B is restored to 2000 and A to 1000, and log records < T₀, A, 1000 >, < T₀, abort > are written out

(b) redo (T₀) and undo (T₁): A and B are set to 950 and 2050 and C is

So, here are some examples, for example, here in (a) undo T₀, B will be restored to, B will be restored to 1000 if you undo, and A will be restored to, I am sorry B will be restored to 2000, A will be restored to 1000 and we will write a log record T₀, B, 2000, T₀, A, 1000 and T₀ abort. So, similarly you can take each case and be convinced that this is how the recording of undo, redo will go.

(Refer Slide Time: 33:43)

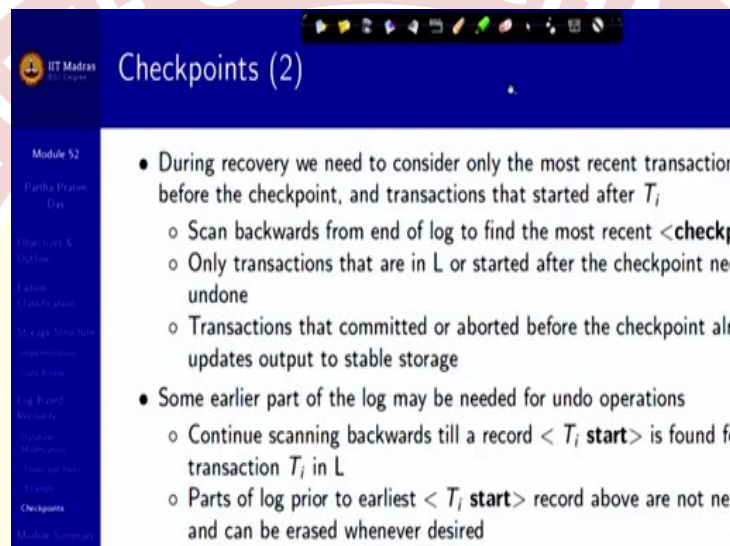
Checkpoints

- Redoing/undoing all transactions recorded in the log can be very slow
 - Processing the entire log is time-consuming if the system has run
 - We might unnecessarily redo transactions which have already output to the database
- Streamline recovery procedure by periodically performing **checkpoint**
- All updates are stopped while doing checkpointing
 - Output all log records currently residing in main memory onto stable storage
 - Output all modified buffer blocks to the disk
 - Write a log record < checkpoint L > onto stable storage where L is the number of transactions active at the time of checkpoint

Now, when you are undoing, redoing and undoing all transactions if you have to do it for the entire log that will be very slow. So, what you do is you create some check points so that you can streamline this recovery process. Check points are like some points in time where you say, this is a check point, where you stop all updates, there is nothing is happening.

So, output all log records currently residing in main memory into stable storage. Output all modified blocks to the disk. And write a log record checkpoint L. So, what you are saying that at this point see this is not possible at every point because you are every moment some changes are happening, so you have to first stop the changes to happen, and then check point to write back everything you had in the buffers at that point, and mark that this is my check point and at that time obviously there will be some transactions which will still be active, so you have to check point is the next that you start.

(Refer Slide Time: 35:06)

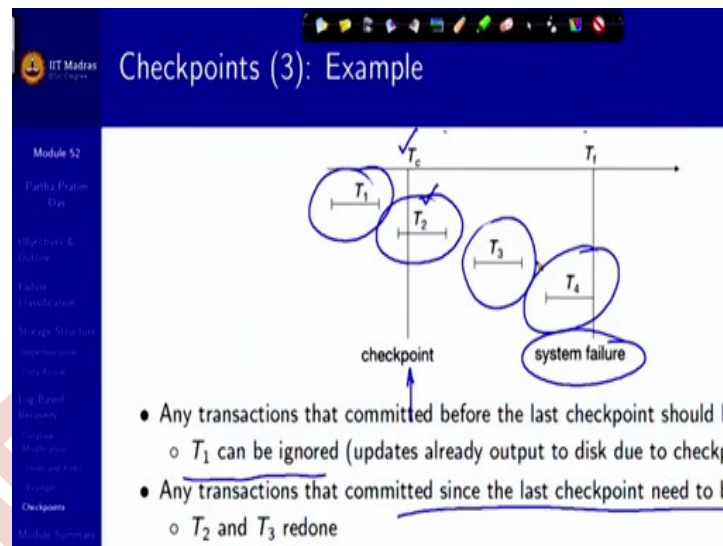


The screenshot shows a presentation slide titled "Checkpoints (2)" from IIT Madras. The slide is part of Module 52, "Faulty System: Disk". The left sidebar lists topics: Overview & Outline, Faulty System, Storage Structure, Replication, Data Access, Log Based Recovery, Checkpoint, and Checkpoint. The main content area contains two bullet points:

- During recovery we need to consider only the most recent transaction before the checkpoint, and transactions that started after T_i
 - Scan backwards from end of log to find the most recent <checkpoint>
 - Only transactions that are in L or started after the checkpoint need to be undone
 - Transactions that committed or aborted before the checkpoint already have their updates output to stable storage
- Some earlier part of the log may be needed for undo operations
 - Continue scanning backwards till a record < T_i start > is found for transaction T_i in L
 - Parts of log prior to earliest < T_i start > record above are not needed and can be erased whenever desired

So, what will this the advantage that check point will give you is when you do recovery you do not have to look at all transactions, you have to look at transactions that started before the check point but completed after that checkpoint or the transactions that started after that checkpoint that will be enough, let us look at an example let us say.

(Refer Slide Time: 35:32)



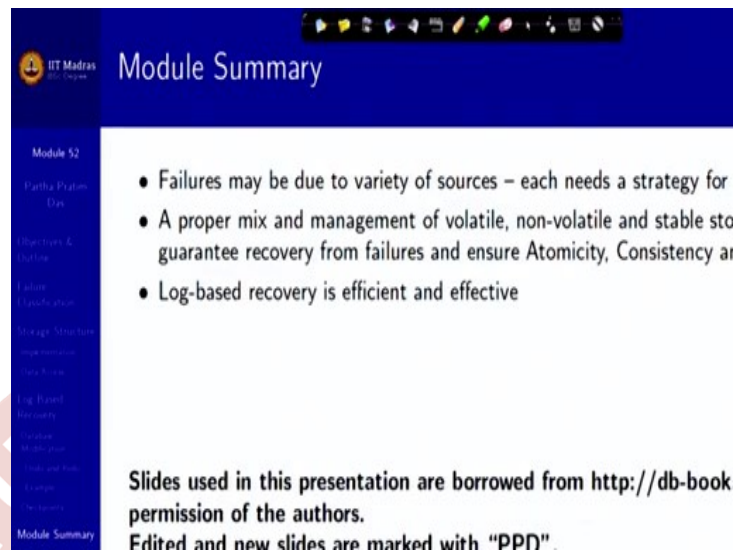
So, you have a check point here, you have another check point here. So, if you, I am sorry this is this is a check point and this is a failure, this is a point at which failure has happened. So, what have you done at this point? You have written down all buffers everything started it and you have kept a track that T_2 was in execution at that time.

So, all transactions have been committed that had completed before the checkpoint. So, T_1 has been committed so it can be ignored, because you already at the check point you have had everything updated in the stable storage. The transactions that are what has failed, this has not been updated it had started but did not finish, it was the L. And this transaction which had started after the checkpoint and finished.

So, these are the transactions which have finished since the last checkpoint before the failure. So, you will have to again perform them that is the need for the redo. Transaction T_4 started after this check point but has not finished, so you have to undo that, because its effects are still not frozen.

So, this is the simple idea of the checkpoint. So, naturally again the cost benefit is to, if you checkpoint very often then your system throughput will go down because you have to stop everything to do the check pointing. If you do checkpoint very sparsely then your transaction logs will become much larger inside and your recovery will become expensive depending it is DBA's responsibility to balance between the two.

(Refer Slide Time: 37:32)



Module Summary

- Failures may be due to variety of sources – each needs a strategy for
- A proper mix and management of volatile, non-volatile and stable storage to guarantee recovery from failures and ensure Atomicity, Consistency and Durability
- Log-based recovery is efficient and effective

Slides used in this presentation are borrowed from <http://db-book> with the permission of the authors.
Edited and new slides are marked with "PPN".

So, that brings us to the end of this module where we have discussed about different types of failures and particularly the use of logs, transaction logs in recovery and illustrated with serial transaction. In the next module we will take this, generalize this to concurrent transactions as well. Thank you very much for your attention. See you in the next module.